

Compliance Module

Pinia Migration — Implementation Master Guide

`src/stores/compliance.js` | `Vue 3 + Pinia` | `grc_frontend`

Engineering Audit | 2026-04-28 | Internal — Not for Distribution

PART 1

Audit & Migration Plan
*What exists, what needs
changing, priority matrix*

PART 2

Store Implementation
*Full state shape, design
decisions, component map*

PART 3

Getters & Actions Blueprint
*Every getter and action with
signatures, TTLs, naming*

PART 1

Audit & Migration Plan

Current state analysis · data flow risks · store recommendations · migration roadmap

1. Module Summary

1.1 What This Module Does

- Control register: Framework → policy → sub-policy → controls (compliances), list/detail/export.
- Organizational controls: Framework-scoped controls, uploads, saves, audit runs (OrganizationalControls.vue).
- Compliance / audit management: Register-style lists, exports (AuditManagementView.vue and related views).
- Approval workflow: Reviewer and final approval queues, detail view, resubmission/rejection (ComplianceApprover.vue, ComplianceDetails.vue).
- Lifecycle: Create, edit, copy, tailoring, versioning (toggle/deactivate flows), baseline configuration.
- Analytics: User compliance dashboard, KPI dashboard (ComplianceDashboard.vue, ComplianceKPINew.vue).
- Other: Cross-framework mapping, tree view and audit routes, internal demo/debug.

1.2 Routes → Components

Route	Name	Component
/compliance/list	AllCompliance	AllCompliance.vue
/compliance/organizational-controls	OrganizationalControls	OrganizationalControls.vue
/compliance/audit-management	AuditManagement	AuditManagementView.vue
/compliance/approver	ComplianceApprover	ComplianceApprover.vue
/compliance/create	CreateCompliance	CreateCompliance.vue
/compliance/tailoring	ComplianceTailoring	ComplianceTailoring.vue
/compliance/versioning	ComplianceVersioning	ComplianceVersioning.vue
/compliance/baseline-configuration	BaselineConfiguration	BaselineConfiguration.vue
/compliance/user-dashboard	ComplianceDashboard	ComplianceDashboard.vue

Route	Name	Component
/compliance/kpi-dashboard	ComplianceKPI	ComplianceKPINew.vue
/compliance/audit-status	Compliances	Compliances.vue
/compliance/audit-status/all	AllComplianceAudit	ComplianceAuditView.vue (lazy)
/compliance/cross-framework-mapping	CrossFrameworkMapping	CrossFrameworkMapping.vue
/compliance/view/:type/:id/:name	ComplianceView	ComplianceView.vue
/compliance/audit/:type/:id/:name	ComplianceAuditView	ComplianceAuditView.vue
/compliance/details/:complianceId	ComplianceDetails	ComplianceDetails.vue
/compliance/edit/:id	EditCompliance	EditCompliance.vue
/compliance/copy/:id	CopyCompliance	CopyCompliance.vue

2. Current Vue Implementation Summary

Topic	Finding
API style	Mix of Options API and <script setup>. AllCompliance.vue, AuditManagementView.vue, ComplianceAuditView.vue, ComplianceView.vue use setup.
Pinia today	ComplianceDashboard.vue uses useDashboardsStore and useAppDataStore. useFrameworkStore exists but many pages still call FRAMEWORK_GET_SELECTED / FRAMEWORK_SET_SELECTED directly.
Caching	Singleton complianceDataService (services/complianceService.js) + window.complianceDataFetchPromise for dedupe — outside Pinia. useAppDataStore.fetchCompliances also uses complianceDataService — two consumers of the same cache pattern.
HTTP	Mostly apiService via local axios shim; axiosInstance in OrganizationalControls.vue; raw axios in CrossFrameworkMapping.vue.
RBAC	AccessUtils in several views; USER_ROLE + string checks (e.g. 'GRC Administrator') in ComplianceDetails.vue / ComplianceApprover.vue.

3. Current Data Flow & Risks

1. Route activated → page mount.
2. Many pages fetch session framework (FRAMEWORK_GET_SELECTED) then hierarchy endpoints.
3. Data stored in component state and/or complianceDataService.dataStore and/or appDataStore.
4. Mutations trigger refetch or navigation — no single invalidation bus.

Current Risks

- Duplicate framework/session fetches across every compliance page.
- Multiple state sources: component state vs complianceDataService vs appDataStore.
- State mismatch between list vs detail vs cache after mutations.
- window.complianceDataFetchPromise — global mutable state, invisible to Pinia DevTools.

4. Variable Inventory (Grouped)

Bucket	Typical Symbols	Keep Local?	Pinia?
Hierarchy lists	frameworks, policies, subpolicies, compliances	No (shared)	Yes — complianceStore
Session / selected framework	sessionFrameworkId, selectedFramework	Align with store	frameworkStore
Export UI	exportFormat, isExporting, dropdown open	Yes	Job id/result optional
Approvals	grouped lists, filters, tabs	Partial UI local	Yes — list + cache metadata
Modals / choosers	showRejectModal, selectedControl, column chooser	Yes	No
Dashboard copy	dashboardData, local frameworks	No	dashboardsStore + actions

5. State Classification

5.1 Keep Local (Component ref / reactive)

Modal visibility, column chooser, export format dropdown, textarea for rejection reason, transient validation, Sidebar openMenus.* for compliance submenus.

5.2 Move to Pinia

Selected framework (backend session), compliance prefetch cache, approval list coherence, dashboard KPI slice.

5.3 API / Server Data → Pinia Actions

Lists, details, export task status, KPI payloads — Pinia actions + optional normalized state; call complianceService / apiService inside actions only.

6. API Inventory (Families)

Central definitions: grc_frontend/src/services/api.js (complianceService), grc_frontend/src/config/api.js (API_ENDPOINTS).

Family	Examples	Used In (Representative)
Framework session	FRAMEWORK_GET_SELECTED, FRAMEWORK_SET_SELECTED	Most compliance pages
Hierarchy	COMPLIANCE_ALL_POLICIES_FRAMEWORKS, policies/subpolicies/compliances by id	AllCompliance, Compliances
Public frameworks	api/compliance/frameworks/public/	ComplianceAuditView, complianceService
Audit management	/api/compliance/all-for-audit-management/, public variant	AuditManagementView, ComplianceAuditView
CRUD / clone	COMPLIANCE_CREATE, COMPLIANCE_UPDATE, clone	CreateCompliance, EditCompliance, CopyCompliance
Detail	complianceService.getComplianceById, COMPLIANCE_GET	ComplianceDetails, copy/edit
Approvals	user/reviewer approval APIs, submit/resubmit	ComplianceApprover, complianceService
Dashboard / KPI	COMPLIANCE_USER_DASHBOARD, KPI endpoints, analytics POST	ComplianceDashboard, ComplianceKPINew, appDataStore
Export	COMPLIANCE_EXPORT, EXPORT_COMPLIANCE_MANAGEMENT, export status URL	Compliances, AllCompliance, AuditManagementView
Versioning	toggle version, deactivate	ComplianceVersioning
Baseline	baseline list/create endpoints	BaselineConfiguration
Org controls	/api/organizational-controls/*, frameworks	OrganizationalControls
Cross-framework	/api/cross-framework-mapping/, compare	CrossFrameworkMapping
RBAC / users	USER_ROLE, USERS_FOR_DROPDOWN, USERS_FOR_REVIEWER_SELECTION	ComplianceDetails, ComplianceApprover, forms
Notifications	PUSH_NOTIFICATION	ComplianceApprover

7. Recommended Pinia Stores (Module-Scoped)

Store	Path	Purpose
frameworkStore (existing)	stores/framework.js	Single source for selected framework + session sync — use everywhere instead of per-page FRAMEWORK_* calls.
complianceStore (NEW)	stores/compliance.js	Hierarchy cache, audit list, approvals, detail-by-id, export jobs, status/lastFetched/TTL, in-action fetch dedupe. Replaces window.complianceDataFetchPromise.

Store	Path	Purpose
dashboardsStore (existing)	stores/dashboards.js	Compliance dashboard KPI cache — single writer from a compliance dashboard action.
appDataStore (existing)	stores/appData.js	Either sole prefetch orchestrator OR delegate to complianceStore — avoid two parallel truths.
permissionStore (NEW)	stores/permission.js	can(permission) aligned with backend RBAC; replace ad-hoc role strings over time.

8. Per-Page Migration Matrix

Page	File	Move to Pinia	Keep Local	Priority
Controls list	AllCompliance.vue	frameworkStore, complianceStore hierarchy + export	Column chooser, export dropdown, breadcrumbs	P0
Org controls	OrganizationalControls.vue	complianceStore or slim org store	Wizard steps, file pickers, inline errors	P1
Audit management	AuditManagementView.vue	complianceStore	Table UI, column visibility	P1
Compliance approval	ComplianceApprover.vue	complianceStore + permissionStore	Tab UI, inline filters	P1
Create	CreateCompliance.vue	Actions + invalidate hierarchy/lists	Entire form model	P1
Edit	EditCompliance.vue	Same	Form fields	P1
Copy	CopyCompliance.vue	Same	Form fields	P1
Tailoring	ComplianceTailoring.vue	frameworkStore, complianceStore	Template UI state	P1
Versioning	ComplianceVersioning.vue	complianceStore	Modals, selection UI	P1
Baseline	BaselineConfiguration.vue	complianceStore.baseline*	Local tree UI	P2
User dashboard	ComplianceDashboard.vue	loadComplianceDashboard → dashboardsStore	Chart display options	P1
KPI	ComplianceKPINew.vue	fetchKpiBundle in complianceStore	Selected chart/tab UI	P1
Audit status	Compliances.vue	complianceStore	Filters, pagination UI	P1
Audit status all	ComplianceAuditView.vue	complianceStore	Filters	P1
Cross-framework	CrossFrameworkMapping.vue	Pinia actions using apiService	Two framework	P2

Page	File	Move to Pinia	Keep Local	Priority
			pickers, diff UI	
View	ComplianceView.vue	complianceStore or thin composable	statusFilter, modal, columns	P2
Details	ComplianceDetails.vue	fetchComplianceDetail, permissionStore	Reject modal, comments	P1

9. Priority Matrix

Priority	Work
P0	frameworkStore everywhere; complianceStore prefetch replacing window.*; pilot AllCompliance.vue
P1	Hierarchy + audit list + approvals in store; mutation invalidation; dashboard/KPI bundle; create/edit/copy/details migration
P2	KPI bundle; export job handling; CrossFrameworkMapping use apiService; baseline + org controls
P3	Component splits; deep normalization; PopupDemo, ComplianceDebug

10. What / Where / Why & Implementation Steps

#	What	Where	Why
W1	Stop duplicating FRAMEWORK_GET_SELECTED / FRAMEWORK_SET_SELECTED	All compliance components	Same framework as HomeView/shell; fewer bugs and duplicate requests
W2	Kill window.complianceDataFetchPromise — move dedupe into complianceStore	AllCompliance, ComplianceDashboard, ComplianceApprover, TT, CopyCompliance, etc.	Testable, visible in Pinia DevTools, no global leaks
W3	Single writer for compliance dashboard cache — loadComplianceDashboard writes dashboardsStore	ComplianceDashboard.vue	Matches existing TTL pattern in stores/dashboards.js

Safe Implementation Order

5. Create complianceStore with status, error, lastFetched, isLoading, isRefreshing.
6. Implement prefetchComplianceDomain (replaces fetchAllComplianceData from components).
7. Align appDataStore.fetchCompliances — one orchestration path only.

8. Migrate `FRAMEWORK_*` reads/writes to `useFrameworkStore` page by page.
9. Move hierarchy fetches to `fetchHierarchy`.
10. Wire mutations to `invalidateAfterComplianceMutation`.
11. Dashboard: single action + `dashboardsStore`.
12. Introduce `permissionStore` incrementally; replace `USER_ROLE` string checks on pilot screens.
13. Regression-test all routes from §1.2.
14. Remove dead globals and duplicate caches.

11. Testing Checklist

- Each route in §1.2 loads without console errors.
- Framework selected on Home matches compliance pages after navigation.
- Cache-first: second visit uses cache within TTL; force refresh refetches.
- Loading vs refreshing UX distinct where applicable.
- Error + retry paths for list, detail, export, KPI.
- Empty hierarchy / empty approvals states render correctly.
- Create → list shows new row (invalidation working).
- Approve/reject → detail + list consistent.
- Export: start, poll, complete.
- RBAC: denied user sees safe UI + 403 handling.
- Logout clears `dashboardsStore` compliance slice + `complianceStore.reset()`.
- Pinia DevTools: expected keys after each flow.

12. Developer Guidelines

Rule	Guidance
1. Local state	Modals, choosers, form fields, page-only filters, sidebar submenu toggles.
2. Pinia	Shared lists, session-aligned framework (<code>frameworkStore</code>), prefetch, approvals, detail cache, dashboard writes to <code>dashboardsStore</code> .
3. API calls	Only inside store actions. Never call API directly from components.
4. Getters	Freshness checks, counts, derived lists.
5. Actions	Fetch, mutate, invalidate, export orchestration.
6. Do NOT store	Huge transient form blobs unless multi-step requirement is explicit.
7. First store work	<code>complianceStore</code> prefetch + <code>frameworkStore</code> adoption.
8. First component	<code>AllCompliance.vue</code> as pilot.

Rule	Guidance
9. Before next module	P0 + P1 checklist green; no duplicate prefetch; framework sync verified.

PART 2

Store Implementation

Full state shape · design decisions · cache template · component usage guide

13. Store File Metadata

Property	Value
Store ID	compliance
File path	src/stores/compliance.js
Export	useComplianceStore
Definition style	Options-style defineStore (state / getters / actions)
Reset support	\$reset() via initialState factory function
Module scope	Entire Compliance sidebar section — all routes in §1.2

14. State Shape Overview

Slice	Key Fields	Purpose
Hierarchy	frameworks, policiesByFrameworkId, subpoliciesByPolicyId, compliancesBySubpolicyId, compliancesById, compliancelds	Full framework → policy → sub-policy → compliance tree
Selection context	selectedFrameworkId, selectedPolicyId, selectedSubpolicyId, selectedComplianceId	Active navigation context shared across pages
Detail / view data	complianceDetailsById, complianceViewData, complianceAuditViewData	Fetched-on-demand detail cache
Approvals	approvalsAsUser, approvalsAsReviewer, rejectedApprovals, approvalFilters	Reviewer and user approval queues + pagination filters
Lists	auditManagementList, auditStatusList, auditInfoByComplianceId	Flat audit lists and per-compliance audit metadata
Shared lookups	reviewers, categoryBusinessUnits	Dropdown options shared by Create/Edit/Copy forms

Slice	Key Fields	Purpose
Baseline	baselineConfigsByFrameworkId, baselineVersionsByFrameworkId	Per-framework baseline data
Cross-framework	crossFramework.*	Framework comparison state — IDs, mappings, diff result
Org controls	organizationalControlsByFrameworkId, organizationalStatsByFrameworkId, organizationalAuditRunsByFrameworkId	Per-framework organizational control data
Dashboard / KPI	dashboardSummary, kpiSummary, kpiCharts	Analytics payloads, also written to dashboardsStore
Export jobs	exportJobs.complianceRegister, exportJobs.auditManagement	In-progress or completed export task metadata
Global async	status, isLoading, isRefreshing, error	Top-level loading/error flags for page-skeleton use
Slice async	loadingMap.*, errorMap.*	Per-slice granular loading and error state
Cache metadata	lastFetched.*	Timestamps used by cache-first TTL logic

15. Complete State Reference (initialState)

Hierarchy + Indexes

```
frameworks: [],
policiesByFrameworkId: {}, // { [frameworkId]: Policy[] }
subpoliciesByPolicyId: {}, // { [policyId]: SubPolicy[] }
compliancesBySubpolicyId: {}, // { [subpolicyId]: Compliance[] }
compliancesById: {}, // { [complianceId]: Compliance } – flat index
complianceIds: [], // string[] – ordered ID list
```

Selection Context

```
selectedFrameworkId: null,
selectedPolicyId: null,
selectedSubpolicyId: null,
selectedComplianceId: null,
```

Detail / View Data

```
complianceDetailsById: {}, // { [complianceId]: ComplianceDetail }
complianceViewData: null,
complianceAuditViewData: null,
```

Approvals

```
approvalsAsUser: [],
approvalsAsReviewer: [],
rejectedApprovals: [],
approvalFilters: { status: 'all', search: '', page: 1, limit: 10 },
```

Lists

```
auditManagementList: [],
auditStatusList: [],
auditInfoByComplianceId: {},
```

Lookups

```
reviewers: [],
categoryBusinessUnits: {
  businessUnitsCovered: [], riskType: [], riskCategory: [], riskBusinessImpact: []
},
```

Baseline

```
baselineConfigsByFrameworkId: {},
baselineVersionsByFrameworkId: {},
```

Cross-Framework

```
crossFramework: {
  frameworks: [], framework1Id: null, framework2Id: null,
  mapping1: null, mapping2: null, comparisonResult: null,
},
```

Organizational Controls

```
organizationalControlsByFrameworkId: {},
organizationalStatsByFrameworkId: {},
organizationalAuditRunsByFrameworkId: {},
```

Dashboard / KPI

```
dashboardSummary: null,
kpiSummary: null,
kpiCharts: {},
```

Export Jobs

```
exportJobs: { complianceRegister: null, auditManagement: null },
```

Async Flags

```
status: 'idle', // 'idle' | 'loading' | 'success' | 'error'
isLoading: false,
isRefreshing: false,
error: null,
```

Slice-Level Maps

```
loadingMap: {
  frameworks, hierarchy, approvals, details, dashboard, kpi,
  baseline, crossFramework, organizationalControls, exports // all: false
},
errorMap: { /* same keys - all: null */ },
```

Cache Metadata

```
lastFetched: {
  frameworks: null, hierarchy: null, approvals: null,
  complianceDetails: {}, // { [complianceId]: timestamp }
  auditManagement: null, dashboard: null, kpi: null,
  baseline: {}, // { [frameworkId]: timestamp }
  crossFramework: null,
  organizationalControls: {}, // { [frameworkId]: timestamp }
  exports: null,
},
```

16. Key Design Decisions

Decision	Rationale
initialState() factory function	Ensures this.\$reset() always clones a fresh copy and avoids shared reference bugs across component instances.
Dual-level async (global + slice)	isLoading/error serve page-skeleton use cases. loadingMap/errorMap allow individual sections to show spinners or retry buttons without blocking the whole page.
Per-key lastFetched for detail/baseline/orgControls	complianceDetails, baseline, and organizationalControls each use { [id]: timestamp } sub-keys because records are fetched independently. markFetched(scope, key) handles this transparently.
crossFramework as nested object	Groups picker state (framework1Id/2Id) with result data (mapping1/2, comparisonResult). Reset via clearCrossFrameworkComparison() or invalidate() fallback.

Decision	Rationale
Export jobs in store	exportJobs tracks task IDs and polling state for async exports. pollExportStatus called on setInterval until 'complete' or 'error'.
compliancesById flat index	Allows O(1) lookup by compliance ID without traversing the hierarchy tree — important for detail pages and approval flows.

17. Cache-First Implementation Template

Copy and adapt this pattern for every fetch action stub:

```
async fetchFrameworks({ force = false } = {}) {
  const TTL = 15 * 60 * 1000 // 15 min – tune per slice
  const fresh = this.lastFetched.frameworks
    && (Date.now() - this.lastFetched.frameworks < TTL)

  if (!force && fresh) return // cache hit – do nothing

  const hasData = this.frameworks.length > 0
  hasData ? (this.isRefreshing = true) : (this.isLoading = true)
  this.markLoading('frameworks')

  try {
    const data = await apiService.get(API_ENDPOINTS.COMPLIANCE_ALL_POLICIES_FRAMEWORKS)
    this.frameworks = data
    this.markFetched('frameworks')
    this.clearScopeError('frameworks')
  } catch (err) {
    this.setScopeError('frameworks', err?.message || 'Failed')
    this.error = err?.message || 'Failed'
  } finally {
    this.isLoading = false
    this.isRefreshing = false
    this.markLoading('frameworks', false)
  }
},
```

TTL Reference

Slice	Recommended TTL	Rationale
frameworks / hierarchy	5–15 minutes	Rarely changes
approvals	30–60 seconds	Changes as reviews progress
complianceDetails	30–60 seconds	Force-refresh after any mutation
auditManagement	2–5 minutes	Moderate frequency
dashboard / kpi	1–5 minutes	Balance freshness vs API load
baseline	5–15 minutes	Per-framework; infrequent changes
crossFramework	5–15 minutes	User-initiated refresh acceptable

Slice	Recommended TTL	Rationale
organizationalControls	5–15 minutes	Per-framework; moderate frequency

18. Component Usage Guide

18.1 Setup in a Component

```
import { storeToRefs } from 'pinia'
import { useComplianceStore } from '@stores/compliance'

const store = useComplianceStore()
const {
  frameworks, policiesForSelectedFramework,
  pendingApprovalCount, isLoading, isRefreshing,
  loadingMap, errorMap,
} = storeToRefs(store)

onMounted(() => {
  store.fetchFrameworks()
  store.fetchApprovalsAsReviewer({ userId: currentUserid })
})
```

18.2 Mutation + Invalidation Pattern

```
async function approveFinding() {
  await apiService.post(API_ENDPOINTS.COMPLIANCE_APPROVE, payload)
  store.invalidate('approvals')
  store.fetchApprovalsAsReviewer({ force: true })
}
```

18.3 Framework Selection Wiring

```
import { useFrameworkStore } from '@stores/framework'
const frameworkStore = useFrameworkStore()

watch(() => frameworkStore.selectedFramework, (fw) => {
  store.setSelectedFramework(fw?.id ?? null)
  store.fetchHierarchy({ frameworkId: fw?.id, force: true })
})
```

NOTE Never call `FRAMEWORK_GET_SELECTED` or `FRAMEWORK_SET_SELECTED` directly from compliance components. Always read from `frameworkStore`.

19. What NOT to Put in the Store

State type	Where it belongs	Examples
Form field values	Component ref / reactive	Create/edit form inputs in CreateCompliance.vue, EditCompliance.vue
Modal open/close flags	Component ref	showRejectModal, showColumnChooser
Rejection reason textarea	Component ref	rejectReason in ComplianceDetails.vue
Export format selection	Component ref	selectedFormat dropdown in AllCompliance.vue
Table column visibility	Component ref (or localStorage)	Column chooser state
Per-page filter UI state	Component ref	Status filter chips, search input (unless cross-route)
Chart display preferences	Component ref	Selected chart tab in KPI / dashboard views

20. Related Stores

Store	File	Relationship
frameworkStore	src/stores/framework.js	Source of truth for selected framework — complianceStore reads from it; never fetches FRAMEWORK_* independently
dashboardsStore	src/stores/dashboards.js	fetchComplianceDashboard writes dashboardsStore.set('compliance', payload) — dashboardsStore is the cache sink
appDataStore	src/stores/appData.js	fetchCompliances must delegate to complianceStore or share the same service helper — avoid two parallel prefetches
permissionStore	src/stores/permission.js	can('...') checks replace direct USER_ROLE string checks in ComplianceApprover, ComplianceDetails

PART 3

Getters & Actions Blueprint

Every getter · every action · signatures · TTLs · naming conventions · implementation order

21. Getters — Complete Blueprint

A. Selection + Context Getters

Getter	Description
activeFrameworkId	Returns selectedFrameworkId
activePolicyId	Returns selectedPolicyId
activeSubpolicyId	Returns selectedSubpolicyId
activeComplianceId	Returns selectedComplianceId
selectedCompliance	Returns compliancesById[selectedComplianceId] or null

B. Hierarchy Data Getters (parameterized)

Getter	Signature	Returns
policiesForFramework	(state) => (frameworkId)	policiesByFrameworkId[frameworkId] []
subpoliciesForPolicy	(state) => (policyId)	subpoliciesByPolicyId[policyId] []
compliancesForSubpolicy	(state) => (subpolicyId)	compliancesBySubpolicyId[subpolicyId] []
complianceById	(state) => (complianceId)	compliancesById[complianceId] null

C. Derived Counters / KPI Helpers

Getter	Description
totalCompliances	Length of compliances array
activeCompliances	Count of compliances with active status

Getter	Description
<code>inactiveCompliances</code>	Count with inactive status
<code>underReviewCompliances</code>	Count with status Under Review
<code>approvedCompliances</code>	Count with status Approved
<code>rejectedCompliances</code>	Count with status Rejected
<code>complianceCountByFramework(frameworkId)</code>	Derived count from framework-linked hierarchy
<code>approvalRate</code>	Derived approval percentage from available approval statuses

D. Approval Flow Getters

Getter	Description
<code>reviewerPendingItems</code>	<code>approvalsAsReviewer</code> where status is Pending / Under Review / null
<code>reviewerCompletedItems</code>	<code>approvalsAsReviewer</code> where status is Approved or Rejected
<code>userPendingApprovals</code>	<code>approvalsAsUser</code> where status is Pending / Under Review
<code>userRejectedApprovals</code>	<code>approvalsAsUser</code> where status is Rejected
<code>canReviewCompliance(complianceId, userId)</code>	Returns true/false from assignment and review status data
<code>approvalItemById(approvalId)</code>	Finds item across all approval arrays by ID

E. Baseline / Cross-Framework / Org Controls Getters

Getter	Description
<code>baselineConfigsForFramework(frameworkId)</code>	<code>baselineConfigsByFrameworkId[frameworkId] []</code>
<code>baselineVersionsForFramework(frameworkId)</code>	<code>baselineVersionsByFrameworkId[frameworkId] []</code>
<code>crossFrameworkComparison</code>	<code>crossFramework.comparisonResult</code>
<code>organizationalControlsForFramework(frameworkId)</code>	<code>organizationalControlsByFrameworkId[frameworkId] []</code>
<code>organizationalStatsForFramework(frameworkId)</code>	<code>organizationalStatsByFrameworkId[frameworkId] null</code>

F. Export Getters

Getter	Description
<code>complianceRegisterExportJob</code>	<code>exportJobs.complianceRegister</code>
<code>auditManagementExportJob</code>	<code>exportJobs.auditManagement</code>
<code>isExportInProgress</code>	True if either export job has status 'running' or 'pending'

Getter	Description
<code>exportDownloadUrl(jobType)</code>	Returns download URL from <code>exportJobs[jobType]</code> if completed

G. Cache Freshness Getters (TTL-based, parameterized)

Getter	Default TTL	Logic
<code>isFrameworksFresh(ttlMs?)</code>	15 min	<code>lastFetched.frameworks</code> within <code>ttlMs</code>
<code>isHierarchyFresh(ttlMs?)</code>	5 min	<code>lastFetched.hierarchy</code> within <code>ttlMs</code>
<code>isApprovalsFresh(ttlMs?)</code>	60 sec	<code>lastFetched.approvals</code> within <code>ttlMs</code>
<code>isDashboardFresh(ttlMs?)</code>	5 min	<code>lastFetched.dashboard</code> within <code>ttlMs</code>
<code>isKpiFresh(ttlMs?)</code>	5 min	<code>lastFetched.kpi</code> within <code>ttlMs</code>
<code>isComplianceDetailFresh(complianceId, ttlMs?)</code>	60 sec	<code>lastFetched.complianceDetails[complianceId]</code> within <code>ttlMs</code>
<code>isBaselineFresh(frameworkId, ttlMs?)</code>	10 min	<code>lastFetched.baseline[frameworkId]</code> within <code>ttlMs</code>
<code>isOrgControlsFresh(frameworkId, ttlMs?)</code>	5 min	<code>lastFetched.organizationalControls[frameworkId]</code> within <code>ttlMs</code>

H. Loading / Error Helpers

Getter	Description
<code>isAnyLoading</code>	True if <code>isLoading</code> , <code>isRefreshing</code> , or any <code>loadingMap</code> entry is true
<code>hasAnyError</code>	True if global error or any <code>errorMap</code> entry is non-null
<code>errorFor(scope)</code>	<code>errorMap[scope]</code> or null
<code>isLoadingScope(scope)</code>	<code>loadingMap[scope]</code> or false

22. Actions — Complete Blueprint

22.1 Local Context / Non-API Actions

Action	Effect
<code>setSelectedFramework(frameworkId)</code>	Sets <code>selectedFrameworkId</code>
<code>setSelectedPolicy(policyId)</code>	Sets <code>selectedPolicyId</code>
<code>setSelectedSubpolicy(subpolicyId)</code>	Sets <code>selectedSubpolicyId</code>
<code>setSelectedCompliance(complianceId)</code>	Sets <code>selectedComplianceId</code>
<code>setApprovalFilters(filtersPatch)</code>	Shallow-merges patch into <code>approvalFilters</code>
<code>resetApprovalFilters()</code>	Resets <code>approvalFilters</code> to <code>initialState</code> defaults

Action	Effect
<code>resetSelection()</code>	Nulls all four <code>selectedFrameworkId / PolicyId / SubpolicyId / ComplianceId</code>

22.2 Cache Utility Actions (Internal — do not call from components)

Action	Effect
<code>markLoading(scope, value)</code>	Sets <code>loadingMap[scope] = value</code>
<code>setScopeError(scope, error)</code>	Sets <code>errorMap[scope] = error</code>
<code>clearScopeError(scope)</code>	Sets <code>errorMap[scope] = null</code>
<code>markFetched(scope, key?)</code>	Sets <code>lastFetched[scope] = Date.now()</code> — uses sub-key for <code>complianceDetails</code> , <code>baseline</code> , <code>orgControls</code>
<code>invalidate(scope)</code>	Clears named slice + resets <code>lastFetched</code> . Scopes: <code>frameworks</code> , <code>hierarchy</code> , <code>approvals</code> , <code>dashboard</code> , <code>kpi</code> , <code>baseline</code> , <code>orgControls</code> , <code>exports</code> . Anything else → full <code>\$reset()</code>
<code>invalidateComplianceDetail(id)</code>	Clears <code>complianceDetailsById[id]</code> and <code>lastFetched.complianceDetails[id]</code>
<code>resetState()</code>	Calls <code>this.\$reset()</code> — use on logout and module exit

22.3 Framework + Hierarchy Fetch Actions

Action	Signature	Populates
<code>fetchFrameworks</code>	<code>({ force? } = {})</code>	<code>frameworks</code> , <code>lastFetched.frameworks</code>
<code>fetchPoliciesByFramework</code>	<code>(frameworkId, { force? } = {})</code>	<code>policiesByFrameworkId[frameworkId]</code>
<code>fetchSubpoliciesByPolicy</code>	<code>(policyId, { force? } = {})</code>	<code>subpoliciesByPolicyId[policyId]</code>
<code>fetchCompliancesBySubpolicy</code>	<code>(subpolicyId, { force? } = {})</code>	<code>compliancesBySubpolicyId[subpolicyId]</code> , <code>compliancesById</code> , <code>complianceIds</code>
<code>fetchHierarchy</code>	<code>({ frameworkId?, policyId?, subpolicyId?, force? })</code>	Orchestrates full chain fetch
<code>prefetchComplianceDomain</code>	<code>({ force? } = {})</code>	Replaces <code>window.complianceDataFetchPromise</code> — prefetches all shared domain data

22.4 Compliance Detail / View Actions

Action	Signature	Populates
<code>fetchComplianceDetails</code>	<code>(complianceId, { force? } = {})</code>	<code>complianceDetailsById[complianceId]</code>
<code>fetchComplianceViewByType</code>	<code>({ type, id, force? })</code>	<code>complianceViewData</code> — route: <code>/compliance/view/:type/:id/:name</code>

Action	Signature	Populates
fetchComplianceAuditViewByType	<code>(({ type, id, force? }))</code>	complianceAuditViewData — route: /compliance/audit/:type/:id/:name
fetchAuditInfoForCompliance	<code>(complianceId, { force? } = {})</code>	auditInfoByComplianceId[complianceId]

22.5 Approval Actions

Action	Signature	Effect
fetchApprovalsAsUser	<code>(({ userId, page, limit, status, force? }))</code>	approvalsAsUser
fetchApprovalsAsReviewer	<code>(({ userId, page, limit, status, force? }))</code>	approvalsAsReviewer
fetchRejectedApprovals	<code>(({ reviewerId, force? }))</code>	rejectedApprovals
submitComplianceReview	<code>(({ approvalId, approved, comments }))</code>	Submits review + calls refreshApprovalsAfterMutation()
resubmitComplianceApproval	<code>(({ approvalId, payload }))</code>	Resubmits rejected approval + invalidates approvals
refreshApprovalsAfterMutation	<code>()</code>	invalidate('approvals') + force-fetch both queues

22.6 CRUD / Mutation Actions

NOTE All mutation actions must call the invalidation scopes shown below on success.

Action	Signature
createCompliance	<code>(payload)</code>
updateCompliance	<code>(({ complianceId, payload }))</code>
cloneCompliance	<code>(({ complianceId, payload }))</code>
toggleComplianceVersion	<code>(complianceId)</code>
deactivateCompliance	<code>(({ complianceId, payload }))</code>
approveComplianceDeactivation	<code>(({ approvalId, payload }))</code>
rejectComplianceDeactivation	<code>(({ approvalId, payload }))</code>

Mutation	Invalidate Scopes
Create / update / clone	invalidate('hierarchy') invalidate('dashboard') invalidate('kpi')
Version toggle / deactivate	invalidate('hierarchy') invalidateComplianceDetail(complianceId) invalidate('dashboard')
Deactivation approve / reject	invalidate('approvals') invalidateComplianceDetail(complianceId)

22.7 Audit Management / Status Actions

Action	Signature	Populates
fetchAuditManagementList	({ frameworkId?, filters?, force? })	auditManagementList
fetchAuditStatusList	({ filters?, force? })	auditStatusList
fetchAllForAuditManagementPublic	({ force? })	auditManagementList (public variant)
refreshAuditManagement	()	invalidate('hierarchy') + fetchAuditManagementList with force

22.8 Dashboard + KPI Actions

Action	Signature	Populates
fetchComplianceDashboard	({ frameworkId?, filters?, force? })	dashboardSummary + dashboardsStore.set('compliance', ...)
fetchComplianceKpi	({ frameworkId?, filters?, force? })	kpiSummary, kpiCharts
fetchKpiBundle	({ frameworkId?, filters?, force? })	Orchestrates fetchComplianceDashboard + fetchComplianceKpi in one call
syncDashboardCacheWithDashboardsStore	()	Optional — writes dashboardSummary into dashboardsStore if not already synced

22.9 Baseline Configuration Actions

Action	Signature	Effect
fetchBaselineConfigs	(frameworkId, { force? })	Populates baselineConfigsByFrameworkId[frameworkId]
createBaselineVersion	(payload)	Creates version + invalidates baseline for frameworkId in payload
fetchBaselineVersions	(frameworkId, { force? })	Populates baselineVersionsByFrameworkId[frameworkId]
refreshBaselineForFramework	(frameworkId)	Force-fetches configs + versions for frameworkId

22.10 Cross-Framework Mapping Actions

Action	Signature	Effect
<code>fetchCrossFrameworkList</code>	<code>(({ force? }))</code>	Populates <code>crossFramework.frameworks</code>
<code>fetchCrossFrameworkMapping</code>	<code>(frameworkId, { side?, force? })</code>	Populates <code>crossFramework.mapping1</code> or <code>mapping2</code>
<code>compareFrameworkVersions</code>	<code>(payload)</code>	Populates <code>crossFramework.comparisonResult</code>
<code>clearCrossFrameworkComparison</code>	<code>()</code>	Nulls <code>comparisonResult</code> , <code>mapping1</code> , <code>mapping2</code> , both IDs

22.11 Organizational Controls Actions

Action	Signature	Effect
<code>fetchOrganizationalControls</code>	<code>(frameworkId , { force? })</code>	Populates <code>organizationalControlsByFrameworkId[frameworkId]</code>
<code>fetchOrganizationalStats</code>	<code>(frameworkId , { force? })</code>	Populates <code>organizationalStatsByFrameworkId[frameworkId]</code>
<code>uploadOrganizationalControls</code>	<code>(formData)</code>	Uploads file + invalidates org controls for <code>frameworkId</code>
<code>saveOrganizationalControls</code>	<code>(payload)</code>	Saves edits + invalidates org controls
<code>runOrganizationalAudit</code>	<code>(payload)</code>	Runs audit + populates <code>organizationalAuditRunsByFrameworkId[id]</code>
<code>refreshOrganizationalControls</code>	<code>(frameworkId)</code>	Force-fetches controls + stats for <code>frameworkId</code>

22.12 Export Actions

Action	Signature	Effect
<code>startComplianceRegisterExport</code>	<code>(payload)</code>	Initiates export + sets <code>exportJobs.complianceRegister</code> with task ID
<code>startAuditManagementExport</code>	<code>(payload)</code>	Initiates export + sets <code>exportJobs.auditManagement</code> with task ID
<code>pollExportStatus</code>	<code>(({ taskId, type }))</code>	Polls API for task status — call on <code>setInterval</code> until 'complete' or 'error'
<code>clearExportJob</code>	<code>(type)</code>	Nulls <code>exportJobs[type]</code> and clears <code>lastFetched.exports</code>

23. Action Naming Convention

Verb Prefix	Use For	Examples
<code>fetch*</code>	Read APIs with cache-first logic	<code>fetchFrameworks</code> , <code>fetchApprovals</code> , <code>fetchComplianceDashboard</code>
<code>create*</code>	POST operations creating a new resource	<code>createCompliance</code> , <code>createBaselineVersion</code>
<code>update*</code>	PUT/PATCH operations	<code>updateCompliance</code>
<code>toggle*</code>	Boolean state flip on server	<code>toggleComplianceVersion</code>
<code>refresh*</code>	Force-reload a slice	<code>refreshApprovalsAfterMutation</code> , <code>refreshBaselineForFramework</code>
<code>invalidate*</code>	Cache invalidation	<code>invalidate('hierarchy')</code> , <code>invalidateComplianceDetail(id)</code>
<code>set*</code>	Local store state assignments	<code>setSelectedFramework</code> , <code>setApprovalFilters</code>
<code>reset*</code>	Restore to initial values	<code>resetApprovalFilters</code> , <code>resetState</code> , <code>resetSelection</code>

24. Recommended Implementation Order

15. Context + cache utility actions (§22.1 and §22.2).
16. `fetchFrameworks` + hierarchy actions (§22.3).
17. Approval actions (§22.5).
18. Detail / view actions (§22.4).
19. Dashboard / KPI actions (§22.8).
20. Mutation actions + invalidation hooks (§22.6).
21. Baseline, cross-framework, org-controls actions (§22.9 – §22.11).
22. Export actions (§22.12).

TIP *This order keeps migration safe — each group can be tested independently before the next is implemented. Run the testing checklist from §11 after each group.*

25. Master Implementation Checklist

	Phase	Task
	P0	<code>complianceStore</code> file created at <code>src/stores/compliance.js</code> with full <code>initialState()</code> , all getters, and all action stubs.
	P0	<code>frameworkStore</code> is the sole source of selected framework — no compliance component calls <code>FRAMEWORK_GET_SELECTED</code> .
	P0	<code>window.complianceDataFetchPromise</code> removed from all components; replaced by <code>prefetchComplianceDomain({ force })</code> .

	Phase	Task
	P0	AllCompliance.vue migrated and working as the pilot consumer.
	P1	All fetch action stubs replaced with real apiService calls following the cache-first template in §17.
	P1	appDataStore.fetchCompliances delegated to / aligned with complianceStore — no parallel cache ownership.
	P1	Approvals slice wired in ComplianceApprover.vue and ComplianceDetails.vue.
	P1	All mutations call invalidate() for the correct scope(s) per §22.6 table.
	P1	dashboardsStore.set('compliance', ...) called from fetchComplianceDashboard action.
	P1	loadingMap / errorMap wired to skeleton / spinner / retry UI in at least two components.
	P2	Baseline, cross-framework, and org controls actions implemented and tested.
	P2	Export polling flow implemented and tested end-to-end.
	P3	permissionStore.can() replaces USER_ROLE string checks in ComplianceApprover and ComplianceDetails.
	P3	resetState() called on logout / module exit.
	ALL	P0+P1 testing checklist from §11 fully green before moving to next module.

End of Document — Compliance Module Pinia Migration Master Guide